

CYPRESS

VS

PLAYWRIGHT

Complete Interview Guide

Installation · Architecture · Commands · CI/CD · Best Practices

QA Automation | Test Engineering | DevOps

Your complete reference for QA & Automation Interviews

*Covers installation, architecture, selectors, assertions, CI/CD pipelines,
parallelism, debugging, best practices and 50+ interview Q&A*

1. Overview & Quick Comparison

What they are, who made them, and how they differ at a glance

What is Cypress?

Cypress is an open-source, JavaScript-only end-to-end testing framework created by **Cypress.io** (founded 2015, acquired by Salesforce 2023). It runs **inside the browser** using a unique proxy architecture, giving it direct DOM access without WebDriver. It bundles test runner, assertion library (Chai), mocking, and time-travel debugging into one tool.

What is Playwright?

Playwright is an open-source, multi-language testing framework created by **Microsoft** (released 2020). Built by the same team that originally wrote Puppeteer, it communicates with browsers via the **Chrome DevTools Protocol (CDP)** and its own browser-specific protocols. Supports TypeScript, JavaScript, Python, Java, and C#.

High-Level Comparison

Feature	Cypress	Playwright
Creator	Cypress.io / Salesforce	Microsoft
Release Year	2017	2020
Languages	JavaScript / TypeScript only	JS, TS, Python, Java, C#
Browser Support	Chrome, Edge, Firefox, Electron	Chrome, Edge, Firefox, Safari (WebKit)
Architecture	In-browser (no WebDriver)	CDP + browser protocols (out-of-process)
Multi-tab Support	■ Limited	■ Full
iFrame Support	Limited (workarounds)	Native & easy
Mobile Testing	Viewport simulation only	Device emulation + real devices via Appium
Parallel Execution	Cypress Cloud (paid) or manual	Built-in (free)
Auto-wait	■ Automatic	■ Automatic
Network Interception	cy.intercept()	page.route() / page.fulfill()
Visual Testing	Via plugins	Built-in screenshot/pixel diff
License	MIT (open-source)	Apache 2.0

2. Installation

From zero to first test

Prerequisites (Both Tools)

- Node.js $\geq 18.x$ (LTS recommended)
- npm ≥ 8 or yarn ≥ 1.22
- Git (for CI/CD pipelines)

Installing Cypress

```
# 1. Create a project
mkdir my-project && cd my-project
npm init -y

# 2. Install Cypress
npm install cypress --save-dev

# 3. Open Cypress Launchpad (first-time setup wizard)
npx cypress open

# 4. Or run headlessly immediately
npx cypress run
```

The first **npx cypress open** launches the Launchpad which scaffolds your project, creates *cypress/* folder with sample specs, and lets you choose E2E or Component Testing.

Cypress Project Structure (after init)

```
my-project/
├── cypress/
│   ├── e2e/           ← your test specs (.cy.js / .cy.ts)
│   ├── fixtures/      ← static JSON test data
│   ├── support/
│   │   ├── commands.js ← custom Cypress commands
│   │   └── e2e.js       ← global hooks (beforeEach etc.)
│   └── downloads/      ← downloaded files during tests
├── cypress.config.js   ← main config file
└── package.json
```

Installing Playwright

```
# 1. Create project & install
npm init -y
npm install @playwright/test --save-dev

# 2. Install browsers (Chromium, Firefox, WebKit)
npx playwright install

# 3. Scaffold config + example tests
npx playwright init
```

```
# 4. Run all tests
npx playwright test

# 5. Open HTML report
npx playwright show-report
```

npx playwright install downloads self-contained browser binaries (~300 MB). You can install only specific browsers: *npx playwright install chromium*.

Playwright Project Structure

```
my-project/
├── tests/           ← test files (*.spec.ts)
├── playwright.config.ts ← main config
├── playwright-report/ ← HTML report output
├── test-results/    ← videos, screenshots, traces
└── package.json
```

TypeScript Setup (Playwright – recommended)

```
npm install typescript @types/node --save-dev
npx tsc --init # creates tsconfig.json

# playwright.config.ts is already TS – just use .spec.ts files
```

3. Configuration Files

cypress.config.js — Key Options

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  e2e: {
    baseUrl: "https://example.com",    // prepended to cy.visit()
    specPattern: "cypress/e2e/**/*.cy.{js,ts}",
    supportFile: "cypress/support/e2e.js",
    viewportWidth: 1280,
    viewportHeight: 720,
    defaultCommandTimeout: 6000,      // ms to retry commands
    requestTimeout: 10000,
    responseTimeout: 30000,
    video: true,
    screenshotOnRunFailure: true,
    retries: { runMode: 2, openMode: 0 },
    env: {
      API_URL: "https://api.example.com",
    },
  },
});
```

playwright.config.ts — Key Options

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  timeout: 30_000,
  retries: 2,
  workers: 4,                                // parallel workers
  reporter: [
    ["html"],
    ["list"],
  ],
  use: {
    baseURL: "https://example.com",
    headless: true,
    screenshot: "only-on-failure",
    video: "retain-on-failure",
    trace: "on-first-retry",
    actionTimeout: 5_000,
    navigationTimeout: 15_000,
  },
  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
    { name: "firefox", use: { ...devices["Desktop Firefox"] } },
    { name: "webkit", use: { ...devices["Desktop Safari"] } },
    { name: "mobile", use: { ...devices["iPhone 13"] } },
  ],
});
```

4. Writing Tests — Syntax & Selectors

Cypress — Basic Test

```
// cypress/e2e/login.cy.ts
describe("Login Flow", () => {
  beforeEach(() => {
    cy.visit("/login");
  });

  it("logs in with valid credentials", () => {
    cy.get('[data-cy="username"]').type("admin");
    cy.get('[data-cy="password"]').type("secret123");
    cy.get('[data-cy="submit"]').click();
    cy.url().should("include", "/dashboard");
    cy.contains("Welcome, Admin").should("be.visible");
  });

  it("shows error for wrong password", () => {
    cy.get('[data-cy="username"]').type("admin");
    cy.get('[data-cy="password"]').type("wrong");
    cy.get('[data-cy="submit"]').click();
    cy.get(".error-msg").should("contain", "Invalid credentials");
  });
});
```

Playwright — Basic Test

```
// tests/login.spec.ts
import { test, expect } from "@playwright/test";

test.describe("Login Flow", () => {
  test.beforeEach(async ({ page }) => {
    await page.goto("/login");
  });

  test("logs in with valid credentials", async ({ page }) => {
    await page.getByTestId("username").fill("admin");
    await page.getByTestId("password").fill("secret123");
    await page.getByTestId("submit").click();
    await expect(page).toHaveURL(/dashboard/);
    await expect(page.getByText("Welcome, Admin")).toBeVisible();
  });

  test("shows error for wrong password", async ({ page }) => {
    await page.getByTestId("username").fill("admin");
    await page.getByTestId("password").fill("wrong");
    await page.getByTestId("submit").click();
    await expect(page.locator(".error-msg"))
      .toContainText("Invalid credentials");
  });
});
```

Selector Strategies — Side by Side

Strategy	Cypress	Playwright
By CSS	<code>cy.get('.btn-primary')</code>	<code>page.locator('.btn-primary')</code>
By data-testid	<code>cy.get('[data-cy="id"]')</code>	<code>page.getByTestId('id')</code>
By text	<code>cy.contains('Submit')</code>	<code>page.getByText('Submit')</code>
By role	<code>cy.get('[role="button"]')</code>	<code>page.getByRole('button', {name:'OK'})</code>
By label	<code>cy.get('#label')</code> (manual)	<code>page.getByLabel('Email')</code>
By placeholder	<code>cy.get('[placeholder]')</code>	<code>page.getByPlaceholder('Email')</code>
XPath	<code>cy.xpath()</code> (plugin)	<code>page.locator('xpath=//button')</code>
nth child	<code>cy.get('li').eq(2)</code>	<code>page.locator('li').nth(2)</code>
Chaining	<code>cy.get('form').find('.input')</code>	<code>page.locator('form').locator('.input')</code>

5. Assertions

Cypress Assertions (Chai + Sinon)

```
// Visibility & existence
cy.get(".modal").should("be.visible");
cy.get(".spinner").should("not.exist");

// Text content
cy.get("h1").should("have.text", "Hello World");
cy.get("p").should("contain", "partial text");

// URL & title
cy.url().should("eq", "https://example.com/dashboard");
cy.title().should("include", "Dashboard");

// Attributes & CSS
cy.get("input").should("have.attr", "placeholder", "Email");
cy.get(".btn").should("have.css", "color", "rgb(255,255,255)");

// Checkbox / disabled state
cy.get("#agree").should("be.checked");
cy.get("#submit").should("be.disabled");

// Count
cy.get("li.item").should("have.length", 5);

// Multiple chained
cy.get(".card")
  .should("be.visible")
  .and("have.class", "active")
  .and("contain", "Featured");
```

Playwright Assertions (expect API)

```
// Visibility & existence
await expect(page.locator(".modal")).toBeVisible();
await expect(page.locator(".spinner")).not.toBeVisible();

// Text content
await expect(page.locator("h1")).toHaveText("Hello World");
await expect(page.locator("p")).toContainText("partial text");

// URL & title
await expect(page).toHaveURL("https://example.com/dashboard");
await expect(page).toHaveTitle(/Dashboard/);

// Attributes & CSS
await expect(page.locator("input")).toHaveAttribute("placeholder", "Email");
await expect(page.locator(".btn")).toHaveCSS("color", "rgb(255,255,255)");

// Checkbox / disabled
await expect(page.locator("#agree")).toBeChecked();
```

```
await expect(page.locator("#submit")).toBeDisabled();

// Count
await expect(page.locator("li.item")).toHaveCount(5);

// Soft assertions (collect all failures)
await expect.soft(page.locator("h1")).toHaveText("Title");
await expect.soft(page.locator(".sub")).toBeVisible();
```

6. Network Interception & API Testing

Cypress — Network Interception

```
// Intercept & stub a GET request
cy.intercept("GET", "/api/users", { fixture: "users.json" }).as("getUsers");
cy.visit("/users");
cy.wait("@getUsers");
cy.get(".user-card").should("have.length", 3);

// Intercept & modify response
cy.intercept("POST", "/api/login", (req) => {
  req.reply({ statusCode: 200, body: { token: "fake-token" } });
}).as("login");

// Wait & assert on request
cy.wait("@login").its("request.body").should("include", { username: "admin" });

// Direct API call
cy.request({ method: "POST", url: "/api/token", body: { user: "admin" } })
  .its("body.token").should("be.a", "string");
```

Playwright — Network Interception & API

```
// Route & fulfill (stub)
await page.route("/api/users", async route => {
  await route.fulfill({
    status: 200,
    contentType: "application/json",
    body: JSON.stringify([{ id: 1, name: "Alice" }]),
  });
});

// Abort a request
await page.route("**/*.png", route => route.abort());

// Wait for a specific API call
const [response] = await Promise.all([
  page.waitForResponse(r => r.url().includes("/api/login") && r.status() === 200),
  page.getByRole("button", { name: "Login" }).click(),
]);
const body = await response.json();
expect(body.token).toBeTruthy();

// Standalone API testing (no browser)
const apiContext = await request.newContext({ baseURL: "https://api.example.com" });
const res = await apiContext.post("/auth", { data: { user: "admin", pass: "secret" } });
expect(res.status()).toBe(200);
```

7. Advanced Features

Multi-tab, iFrames, File Uploads, Hooks

Multi-Tab Testing

Cypress: Does NOT support multi-tab natively. Workaround: spy on window.open, assert href, use cy.stub().

```
// Cypress workaround – test link opens new tab
cy.get("a[target='_blank']").should("have.attr", "href", "/about");
```

Playwright: Full native multi-tab support via context.

```
// Playwright – capture new tab
const [newPage] = await Promise.all([
  page.context().waitForEvent("page"),
  page.getByText("Open New Tab").click(),
]);
await newPage.waitForLoadState();
await expect(newPage).toHaveURL(/about/);
```

iFrame Handling

```
// Cypress (requires cypress-iframe plugin)
cy.frameLoaded("#payment-frame");
cy.iframe("#payment-frame").find("input[name='card']").type("4111111111111111");

// Playwright (native)
const frame = page.frameLocator("#payment-frame");
await frame.getByLabel("Card Number").fill("4111111111111111");
```

File Upload

```
// Cypress
cy.get('input[type="file"]').selectFile("cypress/fixtures/resume.pdf");

// Playwright
await page.locator('input[type="file"]').setInputFiles("./fixtures/resume.pdf");
// Multiple files
await page.locator('input[type="file"]').setInputFiles(["a.pdf", "b.pdf"]);
```

Hooks & Test Lifecycle

Hook	Cypress	Playwright
Before all tests	before() => {}	test.beforeAll(async ({}) => {})
Before each test	beforeEach() => {}	test.beforeEach(async ({page}) => {})
After each test	afterEach() => {}	test.afterEach(async ({page}) => {})
After all tests	after() => {}	test.afterAll(async ({}) => {})
Global setup	cypress/support/e2e.js	globalSetup in playwright.config.ts
Skip test	it.skip()	test.skip()

Focused test	it.only()	test.only()
Conditional skip	cy.skipOn(env)	test.skip(condition, reason)

8. Fixtures, Test Data & Page Object Model

Cypress Fixtures

```
// cypress/fixtures/user.json
{ "name": "Alice", "email": "alice@example.com" }

// In test
cy.fixture("user").then((user) => {
  cy.get("#email").type(user.email);
});
// Or shorthand alias
cy.fixture("user").as("userData");
cy.get("@userData").then(({ email }) => cy.get("#email").type(email));
```

Playwright Test Data

```
// Direct import works fine in TS
import userData from "../fixtures/user.json";

test("fill form", async ({ page }) => {
  await page.getByLabel("Email").fill(userData.email);
});
```

Page Object Model (POM) — Cypress

```
// cypress/pages/LoginPage.ts
export class LoginPage {
  visit() { return cy.visit("/login"); }
  username() { return cy.get('[data-cy="username"]'); }
  password() { return cy.get('[data-cy="password"]'); }
  submit() { return cy.get('[data-cy="submit"]').click(); }
  errorMsg() { return cy.get(".error-msg"); }
}

// In test
import { LoginPage } from "../pages/LoginPage";
const login = new LoginPage();
login.visit();
login.username().type("admin");
login.password().type("pass");
login.submit();
```

Page Object Model — Playwright

```
// pages/LoginPage.ts
import { Page, Locator } from "@playwright/test";

export class LoginPage {
  readonly page: Page;
  readonly username: Locator;
  readonly password: Locator;
```

```
readonly submit: Locator;

constructor(page: Page) {
  this.page      = page;
  this.username  = page.getByTestId("username");
  this.password  = page.getByTestId("password");
  this.submit    = page.getByRole("button", { name: "Login" });
}

async login(user: string, pass: string) {
  await this.username.fill(user);
  await this.password.fill(pass);
  await this.submit.click();
}
}

// In test
import { LoginPage } from "../pages/LoginPage";
const loginPage = new LoginPage(page);
await loginPage.login("admin", "secret");
```

9. CI/CD Integration

GitHub Actions · GitLab CI · Jenkins · Docker

GitHub Actions — Cypress

```
# .github/workflows/cypress.yml
name: Cypress E2E Tests

on: [push, pull_request]

jobs:
  cypress-run:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [chrome, firefox]    # Run on multiple browsers

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"

      - name: Install dependencies
        run: npm ci

      - name: Run Cypress tests
        uses: cypress-io/github-action@v6
        with:
          browser: ${ matrix.browser }
          start: npm start             # start dev server
          wait-on: "http://localhost:3000"
          wait-on-timeout: 60
          record: true                 # Cypress Cloud recording
        env:
          CYPRESS_RECORD_KEY: ${ secrets.CYPRESS_RECORD_KEY }

      - name: Upload screenshots on failure
        uses: actions/upload-artifact@v4
        if: failure()
        with:
          name: cypress-screenshots
          path: cypress/screenshots
```

GitHub Actions — Playwright

```
# .github/workflows/playwright.yml
name: Playwright E2E Tests

on: [push, pull_request]
```

```

jobs:
  playwright-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: "npm"

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright Browsers
        run: npx playwright install --with-deps

      - name: Run Playwright tests
        run: npx playwright test
        env:
          BASE_URL: https://staging.example.com

      - name: Upload HTML Report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 14

```

GitLab CI — Playwright

```

# .gitlab-ci.yml
stages: [test]

playwright:
  image: mcr.microsoft.com/playwright:v1.44.0-jammy
  stage: test
  script:
    - npm ci
    - npx playwright test --reporter=junit
  artifacts:
    when: always
    paths: [playwright-report/, test-results/]
    reports:
      junit: test-results/results.xml
  cache:
    key: $CI_COMMIT_REF_SLUG
    paths: [node_modules/]

```

Docker — Running Tests in Containers

```
# Dockerfile (Playwright)
FROM mcr.microsoft.com/playwright:v1.44.0-jammy
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]
```

```
# docker-compose.yml
version: "3.8"
services:
  playwright:
    build: .
    environment:
      - BASE_URL=http://app:3000
    depends_on: [app]
  app:
    image: my-app:latest
    ports: ["3000:3000"]
```

Parallelism in CI

Aspect	Cypress	Playwright
Free parallelism	■ (only Cypress Cloud paid)	■ Built-in (workers config)
Config option	N/A (manual splitting)	workers: 4 in playwright.config.ts
Sharding (CI matrix)	Manual spec splitting	npx playwright test --shard=1/4
Cypress Cloud	\$0–\$399/mo for smart orchestration	Not applicable
GitHub matrix trick	Split specs manually across jobs	Use --shard=N/total

Playwright Sharding Example (GitHub Actions)

```
jobs:
  test:
    strategy:
      matrix:
        shardIndex: [1, 2, 3, 4]
        shardTotal: [4]
    steps:
      - run: npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal
}}
```

10. Debugging

Cypress Debugging

- **Time-Travel Debugger:** hover over commands in Cypress UI to see DOM snapshots at each step.
- **cy.pause():** pause test execution and interact with the app manually.
- **cy.debug():** drops into browser DevTools debugger at that point.
- **cy.log('message'):** prints to command log without opening DevTools.
- **Screenshot on failure:** automatic when running headlessly (cypress run).
- **Videos:** full video of each spec recorded during cy run.

```
cy.get(".btn")
  .debug()           // breakpoint here
  .click();
```

Playwright Debugging

- **Playwright Inspector:** `npx playwright test --debug` — opens step-by-step GUI.
- **Trace Viewer:** `npx playwright show-trace trace.zip` — timeline, DOM snapshots, network.
- **page.pause():** stops execution, opens inspector.
- **PWDEBUG=1:** env var to auto-open inspector.
- **Codegen:** `npx playwright codegen https://example.com` — generates test code by recording actions.

```
# Open inspector on every test
PWDEBUG=1 npx playwright test

# Record & generate test code
npx playwright codegen https://example.com --save-storage=auth.json
```

11. Visual & Accessibility Testing

Visual Testing

Cypress relies on third-party plugins like *@applitools/eyes-cypress* or *cypress-image-snapshot* for pixel-diff visual tests.

```
// cypress-image-snapshot
cy.get(".hero-banner").matchImageSnapshot("hero-banner");
```

Playwright has built-in screenshot comparison via `toHaveScreenshot()`:

```
// Playwright built-in visual comparison
await expect(page).toHaveScreenshot("homepage.png", { maxDiffPixels: 100 });
await expect(page.locator(".chart")).toHaveScreenshot("chart.png");

// Update baselines
npx playwright test --update-snapshots
```

Accessibility Testing

Both tools integrate with **axe-core** for a11y audits:

```
// Cypress – cypress-axe
import "cypress-axe";
cy.visit("/");
cy.injectAxe();
cy.checkA11y(null, { includedImpacts: ["critical", "serious"] });

// Playwright – @axe-core/playwright
import { checkA11y, injectAxe } from "axe-playwright";
await injectAxe(page);
await checkA11y(page, null, { includedImpacts: ["critical"] });
```

12. Architecture Deep-Dive

How they talk to browsers

Cypress Architecture

- **In-browser execution:** Cypress test code runs in the SAME browser process as the app under test — no network latency between test commands and DOM.
- **Node.js proxy server:** sits between browser and server; intercepts/modifies network traffic; enables `cy.intercept()`.
- **No WebDriver:** avoids the W3C WebDriver protocol overhead, making it faster for DOM operations.
- **Limitation:** same-origin restriction means it cannot easily test multiple tabs or cross-origin iframes natively.
- **Electron shell:** the Cypress runner itself is an Electron app for the GUI; tests can also run in Chrome/Edge/Firefox but the Electron one is default for headless.

Playwright Architecture

- **Out-of-process:** Playwright communicates with browsers via WebSocket + Chrome DevTools Protocol (CDP) for Chromium; similar protocols for Firefox and WebKit.
- **Browser contexts:** lightweight isolated browser sessions (cookies, storage separated) — like incognito but scriptable. Fixtures use a fresh context per test.
- **Multi-browser from ground up:** each browser (Chromium, Firefox, WebKit) has tailored protocol bindings — not a WebDriver wrapper.
- **Parallel by design:** each worker process gets its own browser instance; no shared state.
- **No same-origin restriction:** can handle multi-domain flows, cross-origin iframes, multi-tab windows natively.

13. Performance & Speed

Metric	Cypress	Playwright
Test startup time	~3–5 sec (browser launch)	~1–2 sec (reuses browser context)
Command speed	Fast (in-browser)	Fast (WebSocket to browser)
Parallelism	Paid or manual sharding	Free, built-in workers
Browser download size	~300 MB (bundled Electron+browsers)	~280 MB (browser binaries)
Memory per worker	Higher (full browser per spec)	Lower (shared browser, isolated context)
CI minutes (100 tests)	~15 min single thread	~4 min with 4 workers

14. When To Choose Which Tool

Choose Cypress When...

- Your team is JS/TS only and you want the fastest onboarding.
- You love time-travel debugging and the interactive GUI.
- You're testing a SPA (React/Vue/Angular) with a single domain.
- You want built-in component testing alongside E2E.
- Your team is willing to pay for Cypress Cloud for analytics & parallelism.

Choose Playwright When...

- You need multi-browser coverage including Safari/WebKit on Linux CI.
- Tests require multi-tab, popup windows, or cross-origin iFrames.
- Your team uses Python, Java, or C# (or a mix of languages).
- You need free parallel execution at scale.
- You want built-in trace viewer, codegen, and network HAR recording.
- You're testing mobile web with device emulation.

Quick Decision Matrix

Scenario	Recommended
Small team, JS only, SPA	Cypress
Multi-browser incl. Safari	Playwright
Cross-origin flows / multi-tab	Playwright
Budget for paid CI features	Cypress Cloud
Free CI parallelism required	Playwright
Python/Java backend team	Playwright
Component testing alongside E2E	Cypress
Complex trace/debugging tooling	Playwright
Quick interactive demo/POC	Cypress (easier GUI)
Enterprise with strict browser matrix	Playwright

15. Top 30 Interview Questions & Answers

Q1. What is the main architectural difference between Cypress and Playwright?

Cypress runs test code INSIDE the browser process (no WebDriver), giving direct DOM access but limiting cross-origin support. Playwright runs OUT of process, communicating via CDP/WebSocket, allowing full multi-tab, cross-origin, and multi-browser control.

Q2. Does Cypress support Safari?

No. Cypress supports Chrome, Edge, Firefox, and Electron — but NOT Safari/WebKit. Playwright supports all four including Safari via its WebKit engine.

Q3. How does auto-waiting work in both tools?

Both tools automatically retry commands/assertions until they pass or timeout. Cypress retries its entire command chain; Playwright retries locator-based actions and assertions independently with configurable timeouts.

Q4. How do you handle authentication in Playwright across tests?

Use `storageState`: save logged-in cookies/localStorage to a JSON file (`page.context().storageState()`), then reuse it in `playwright.config.ts` or specific test fixtures via use: `{ storageState: 'auth.json' }`.

Q5. What is `cy.intercept()` used for?

To intercept HTTP requests made by the app, stub responses (return fake data), modify requests/responses, or assert what the app sent. This is Cypress's built-in network layer without needing a separate mock server.

Q6. What are Playwright fixtures?

Fixtures are dependency-injected helpers available in test functions (e.g., `page`, `browser`, `context`). You can define custom fixtures to share setup code — like a logged-in page — across many tests without repeating `beforeEach`.

Q7. How does parallelism differ?

Playwright has FREE built-in parallelism via the `'workers'` config option and `'--shard'` for CI matrix splits. Cypress parallel execution requires Cypress Cloud (paid) unless you manually split specs across CI jobs.

Q8. What is the Playwright Trace Viewer?

A timeline UI (`npx playwright show-trace`) that shows every action, DOM snapshot before/after each step, network requests, console logs, and screenshots — like DevTools time-travel. Invaluable for debugging CI failures.

Q9. Can Cypress handle multi-tab testing?

Not natively. Cypress doesn't support multi-tab. The workaround is to stub `window.open` or test the target URL separately. Playwright handles multi-tab with `context.waitForEvent('page')`.

Q10. What is Page Object Model and why use it?

POM is a design pattern that wraps page interactions in reusable classes. Benefits: reduces duplication, centralizes selectors (one place to update when UI changes), and makes tests more readable. Both tools support it equally well.

Q11. How do you run only specific tests in Playwright?

Use `test.only()` to run one test, or pass `--grep 'keyword'` on CLI. Use `--project` to run only one browser. For files: `npx playwright test tests/login.spec.ts`.

Q12. What is `cy.task()` in Cypress?

`cy.task()` lets you run Node.js code from within a test (e.g., seed a database, read a file, call an API). It bridges the browser context and the Node process.

Q13. Explain soft assertions in Playwright.

`expect.soft()` collects assertion failures without stopping the test. All soft assertion failures are reported together at the end. Useful to audit multiple UI properties in one test run without aborting on first fail.

Q14. How do you generate tests automatically in Playwright?

Use `npx playwright codegen`. It opens a browser, records your clicks/types, and generates test code in real-time. You can choose the target language (JS, TS, Python, Java, C#).

Q15. What are the differences in reporter options?

Cypress: spec (default), json, junit, teamcity, mochawesome (plugin). Playwright: list, dot, line, html (built-in), junit, json, allure (plugin) — multiple reporters can run simultaneously.

Q16. How do you handle flaky tests?

Both tools support retries (retries: 2 in config). Playwright also offers `test.fixme()` to mark known flaky. For root-cause: use traces, increase timeouts selectively, ensure stable test data, avoid fixed sleeps — use `auto-wait`.

Q17. What is globalSetup in Playwright?

A file exported from `playwright.config.ts` `globalSetup` that runs ONCE before all test workers start. Used for: creating auth tokens, seeding databases, starting mock servers. Pair with `globalTeardown` for cleanup.

Q18. How do you test file downloads?

Playwright: `const [download] = await Promise.all([page.waitForEvent('download'), page.click('#download-btn')]); await download.saveAs('./output/file.pdf');` Cypress: uses `cy.readFile()` after clicking the download button.

Q19. What is the difference between `page.locator()` and `page.$()` in Playwright?

`page.locator()` returns a `Locator` — a lazy, retrievable reference with built-in auto-waiting. `page.$()` returns a single `ElementHandle` immediately (no auto-wait). Always prefer `locator()` for robust tests.

Q20. Can Playwright test mobile apps?

Playwright tests mobile WEB via device emulation (viewport, user-agent, touch events). For native mobile apps, Playwright itself doesn't apply — use Appium. However, you can combine Playwright for web portions of hybrid apps.

Q21. How do you skip tests conditionally in Cypress?

Use the `cypress-skip-test` plugin or check environment: `if (Cypress.env('CI')) { this.skip(); }`. Or tag tests and run only tagged ones via `cypress-grep` plugin.

Q22. What is Cypress Component Testing?

Cypress can mount individual React/Vue/Angular components in isolation (without full browser navigation) and test them directly. It's similar to React Testing Library but with the full Cypress command API and time-travel.

Q23. Explain Playwright's browser contexts.

A `BrowserContext` is an isolated session within a Browser — its own cookies, `localStorage`, and network state. Multiple contexts can run in ONE browser instance simultaneously, keeping tests independent while sharing the browser process (faster).

Q24. How do you handle environment variables in CI?

Cypress: set via `CYPRESS_VAR_NAME` env vars or `cypress.env.json`. Playwright: standard `process.env.VAR_NAME` in config. In GitHub Actions use secrets: `env: { BASE_URL: ${{ secrets.STAGING_URL }}` `}`.

Q25. What is the difference between `cypress run` and `cypress open`?

`cypress run`: headless mode, no GUI, outputs to terminal — used in CI. `cypress open`: launches the Electron-based interactive runner with time-travel debugger — used during development.

Q26. How do you measure test coverage with these tools?

Cypress: use the `@cypress/code-coverage` plugin (instruments code with Istanbul/ NYC). Playwright: use V8 coverage API (`page.coverage.startJSCoverage()`) or integrate with Istanbul. Both generate coverage reports for CI.

Q27. What is the Cypress Dashboard / Cloud?

Cypress Cloud (formerly Dashboard) is Cypress's paid SaaS. Features: test recordings, smart orchestration for parallelism, flaky test analytics, GitHub PR comments with test results, visual review. Starts ~\$0 for small teams, up to \$399+/mo.

Q28. How do you handle authentication tokens across Playwright tests?

In `globalSetup`, log in once and save state: `await page.context().storageState({ path: 'auth.json' })`. In `playwright.config.ts`: use: `{ storageState: 'auth.json' }`. All tests start with the saved session — no re-login per test.

Q29. What browsers are supported by each tool?

Cypress: Chromium-based (Chrome, Edge), Firefox, Electron — NO Safari. Playwright: Chromium (Chrome, Edge), Firefox, WebKit (Safari engine) — all on Linux, macOS, Windows.

Q30. Give a real scenario where Playwright wins over Cypress.

Payment flow with 3D Secure: the bank opens in a NEW TAB (cross-origin) for OTP. Cypress cannot handle this natively. Playwright natively captures the new page event, interacts with the 3DS iframe, and continues the flow seamlessly.

16. Quick Cheat Sheet

Most-used commands at a glance

Action	Cypress	Playwright
Visit URL	<code>cy.visit('/path')</code>	<code>await page.goto('/path')</code>
Find element	<code>cy.get('.sel')</code>	<code>page.locator('.sel')</code>
Click	<code>cy.get('.btn').click()</code>	<code>await page.locator('.btn').click()</code>
Type text	<code>cy.get('input').type('text')</code>	<code>await page.fill('input', 'text')</code>
Press key	<code>cy.get('input').type('{enter}')</code>	<code>await page.keyboard.press('Enter')</code>
Clear input	<code>cy.get('input').clear()</code>	<code>await page.locator('input').clear()</code>
Get text	<code>cy.get('p').invoke('text')</code>	<code>await page.locator('p').innerText()</code>
Assert visible	<code>.should('be.visible')</code>	<code>await expect(loc).toBeVisible()</code>
Assert text	<code>.should('have.text', 'x')</code>	<code>await expect(loc).toHaveText('x')</code>
Assert URL	<code>cy.url().should('include', '/x')</code>	<code>await expect(page).toHaveURL(/x/)</code>
Wait for API	<code>cy.wait('@alias')</code>	<code>await page.waitForResponse(url)</code>
Take screenshot	<code>cy.screenshot('name')</code>	<code>await page.screenshot({path: 'x.png'})</code>
Select dropdown	<code>cy.get('sel').select('val')</code>	<code>await page.selectOption('sel', 'val')</code>
Hover	<code>cy.get('.x').trigger('mouseover')</code>	<code>await page.locator('.x').hover()</code>
Scroll	<code>cy.get('.x').scrollIntoView()</code>	<code>await page.locator('.x').scrollIntoViewIfNeeded()</code>
Drag & Drop	<code>cy.drag plugin</code>	<code>await page.dragAndDrop('#src', '#tgt')</code>
Reload page	<code>cy.reload()</code>	<code>await page.reload()</code>
Go back	<code>cy.go('back')</code>	<code>await page.goBack()</code>
Execute JS	<code>cy.window().then(w=>w.eval('...'))</code>	<code>await page.evaluate('...')</code>
Wait fixed ms	<code>cy.wait(2000)</code>	<code>await page.waitForTimeout(2000)</code>
Intercept request	<code>cy.intercept('GET', '/api', {})</code>	<code>await page.route('/api', r=>r.fulfill({}))</code>
Read cookie	<code>cy.getCookie('name')</code>	<code>await page.context().cookies()</code>
Set viewport	<code>cy.viewport(1280, 720)</code>	<code>await page.setViewportSize({w, h})</code>

Interview Tip: Be ready to compare based on concrete project needs. Cypress = great DX for JS-only SPAs. Playwright = power + multi-browser + multi-language. Know both; show you can choose the right tool for the right context.

Created with ❤️ for QA Engineers | Cypress vs Playwright Interview Guide 2024–2025